INTERACTIVE RAYMARCHER

Advanced Procedural Volumetrics

Official User Manual & Documentation

Table of Contents

1. Introduction to Raymarching

2. Architecture & Setup

3. The Noise Compute Library

4. Geometry & Displacement

5. Materials & Shading Techniques

6. C# Controllers & Interaction

7. Advanced Effects (Orbs & Haze)

8. Camera Setup (RaymarchCamera)

9. Performance Considerations

1. Introduction to Raymarching

Welcome to the **Interactive Raymarcher** package. Traditional rendering in Unity uses rasterization, where 3D polygons (triangles) are projected onto a 2D screen. While efficient, rasterization struggles to create complex, infinitely smooth, or mathematically morphing shapes like liquids, fractal terrain, or true volumetrics.

Raymarching is a completely different rendering technique. Instead of drawing triangles, the camera shoots a "ray" for every single pixel on your screen. This ray marches forward in steps, constantly checking a mathematical formula called a **Signed Distance Function (SDF)**. The SDF tells the ray exactly how far away it is from the nearest surface. When the distance hits zero (or close to it), we have a "hit," and we draw a pixel.

> **The True Power of SDFs:** Because shapes are defined by math, not polygons, they can be blended together flawlessly. The `smin()` (Smooth Minimum) function allows two separate spheres to merge like water droplets, creating organic, gooey, or surreal geometry that is impossible with standard meshes.

2. Architecture & Setup

This package is built around a collection of custom HLSL shaders that perform raymarching inside a standard Unity bounding box (like a cube or a post-processing quad), alongside C# scripts to feed real-time transform data (position, rotation, scale) into those shaders.

Basic Shader Structure

All shaders in this package share a common architecture designed for maximum performance:

- **MAX_STEPS (100):** The maximum number of times a ray will step forward before giving up.
- **MAX_DIST (100):** The maximum distance (in Unity units) a ray will travel.
- **SURF_DIST (1e-3):** The threshold distance to consider a "hit" (0.001 units).

The core logic resides in the `rayMarch()` function, which iterates through steps, calling `getDist()` to evaluate the complex scene geometry at every step.

3. The Noise Compute Library

Procedural shapes require high-quality noise. This package includes a comprehensive, heavily optimized noise library (`noise.cginc`) ported specifically for real-time raymarching.

Available Noise Types

Function	Description
gnoise (Gradient Noise)	Standard smooth noise, similar to Perlin. Great for gentle undulations and terrain.
snoise (Simplex Noise)	A faster, artifact-free alternative to Perlin noise. Excellent for 3D and 4D volumetric textures.
voronoi (Cellular Noise)	Creates cellular, organic structures like scales, cracked earth, or biological tissue.
fbm (Fractional Brownian Motion)	Layers multiple octaves of noise together to create highly detailed, fractal-like roughness (clouds, mountains).

These noise functions are extensively used to displace spheres and create the "Gyroid" structures seen throughout the package.

4. Geometry & Displacement

Rather than modeling in Blender or Maya, shapes are constructed purely via mathematical functions in the shader.

Core Primitives

- **sphere(p, r):** Defines a perfect sphere at point p with radius `r`.
- **sdBox(p, b):** Defines a box with bounds b.
- **torus(p):** Defines a donut shape.

The Gyroid

A recurring theme in this package is the **Gyroid** (an infinitely connected periodic minimal surface). It looks like complex alien coral or a sponge.

float gyroid = abs(0.7 * dot(sin(p), cos(p.yzx)) / rescaleFactor) - thickness;

By subtracting (using `smin(..., -k)`) this infinite gyroid field from a standard sphere, we create the intricate, hollowed-out "orbs" that pulse with energy.

5. Materials & Shading Techniques

Once a ray hits a mathematical surface, we need to color it. Because there are no UV coordinates in pure math, shading requires clever techniques.

Normal Calculation

We calculate surface normals dynamically using a technique called "central differences." We sample the SDF at points infinitesimally close to the hit position (X, Y, Z) to determine the slope of the surface.

Lighting Models

- **Standard Lighting:** Uses Unity's `_WorldSpaceLightPos0` to calculate standard diffuse (Lambertian) lighting.
- **MatCap (Material Capture):** Shaders like `MatcapSphere` use a pre-rendered 2D sphere texture. By mapping the calculated surface normal to the UVs of the MatCap texture, we achieve incredibly cheap, highly realistic reflections without needing reflection probes or raytracing.
- **Fresnel Glow:** Calculates the dot product of the normal and the view direction to create a glowing rim effect on grazing angles, essential for the "holographic" or "energy" aesthetic.

6. C# Controllers & Interaction

Raymarched shapes are inherently static unless told otherwise. To make them interactive and respond to Unity's physics engine, we use C# scripts to bridge the gap.

DynamicGlowingOrbs.cs

This script manages a group of standard Unity GameObjects (which have Rigidbody and SphereCollider components). In its `Update()` loop, it reads the positions, scales, and rotations of these objects and pushes them into arrays within the Raymarch shader.

This allows the mathematical SDF spheres to perfectly overlay the physical Unity colliders. When the rigidbodies bounce off walls (as seen in `Orb.cs`), the gooey raymarched visuals follow them seamlessly.

7. Advanced Effects (Orbs & Haze)

The `OrbsEnvironmentLighting` and `DualGlowingOrbs` shaders represent the pinnacle of this package.

Subsurface Scattering (SSS)

To make the orbs look like glowing jelly rather than solid plastic, we simulate light penetrating the surface. The `subSurfaceScattering` function calculates how deep the camera is looking into the volume and adds the `_GlowColor` accordingly. The deeper the view, the brighter the glow.

Environment Haze (Depth Fog)

In `OrbsEnvironmentLighting`, we calculate the distance from the camera to the raymarched hit point. If the point is far away (approaching `_HazeMaxDist`), we blend the final color with the `_HazeColor`. This grounds the floating math shapes into the Unity environment, providing essential depth cues.

8. Camera Setup (RaymarchCamera)

While many shaders in this package run on standard meshes (like a cube bounding box), full-screen effects require the `RaymarchCamera.cs` script.

This script attaches to your Main Camera. It calculates the camera's view frustum (the corners of the screen in world space) and passes these vectors to the shader. The shader (e.g., `BaseRaymarcher`) then interpolates between these corners to shoot rays precisely into the scene, rendering the SDFs as a post-processing overlay across the entire screen.

9. Performance Considerations

Raymarching is a GPU-intensive operation. Evaluating complex noise functions (like FBM or Voronoi) inside a 100-step loop for every pixel on a 4K monitor can bring a weak GPU to a halt.

Optimization Tips:

- **Limit Bounding Boxes:** If applying a raymarch shader to a Unity Cube, make the cube as small as possible. Empty space inside the cube still requires ray steps.
- **Reduce Max Steps:** If your geometry isn't highly detailed, lowering `MAX_STEPS` from 100 to 50 or 32 can dramatically increase FPS.
- **Avoid High-Octane Noise:** FBM noise is beautiful but slow. Use simple Sin/Cos math (like the Gyroid function) for organic shapes where possible.